

Exception Handling and Event Handling

- 14.1** Introduction to Exception Handling
- 14.2** Exception Handling in C++
- 14.3** Exception Handling in Java
- 14.4** Exception Handling in Python and Ruby
- 14.5** Introduction to Event Handling
- 14.6** Event Handling with Java
- 14.7** Event Handling in C#

This chapter discusses programming language support for two related parts of many contemporary programs: exception handling and event handling. Both exceptions and events occur at times that cannot be predetermined, and both are best handled with special language constructs and processes. Some of these constructs and processes—for example, propagation—are similar for exception handling and event handling.

We first describe the fundamental concepts of exception handling, including hardware- and software-detectable exceptions, exception handlers, and the raising of exceptions. Then, the design issues for exception handling are introduced and discussed, including the binding of exceptions to exception handlers, continuation, and default handlers. This section is followed by descriptions and evaluations of the exception-handling facilities of two programming languages: C++ and Java. Brief introductions to exception handling in Python and Ruby are then presented.

The latter part of this chapter is about event handling. We first present an introduction to the basic concepts of event handling. This is followed by discussions of the event-handling approaches of Java and C#.

14.1 Introduction to Exception Handling

Most computer hardware systems are capable of detecting certain run-time error conditions, such as floating-point overflow. Early programming languages were designed and implemented in such a way that the user program could neither detect nor attempt to deal with such errors. In these languages, the occurrence of such an error simply causes the program to be terminated and control to be transferred to the operating system. The typical operating system reaction to a run-time error is to display a diagnostic message, which may be meaningful and therefore useful, or highly cryptic. After displaying the message, the program is terminated.

In the case of input and output operations, however, the situation is somewhat different. For example, a Fortran `Read` statement can intercept input errors and end-of-file conditions, both of which are detected by the input device hardware. In both cases, the `Read` statement can specify the label of some statement in the user program that deals with the condition. In the case of the end-of-file, the condition obviously is not always considered an error. In most cases, it is nothing more than a signal that one kind of processing is completed and another kind must begin. In spite of the obvious difference between end-of-file and events that are always errors, such as a failed input process, Fortran handles both situations with the same mechanism. Consider the following Fortran `Read` statement:

```
Read(Unit=5, Fmt=1000, Err=100, End=999) Weight
```

The `Err` clause specifies that control is to be transferred to the statement labeled 100 if an error occurs in the read operation. The `End` clause specifies that control is to be transferred to the statement labeled 999 if the read

operation encounters the end of the file. So, Fortran uses simple branches for both input errors and end-of-file.

There is a category of serious errors that are not detectable by hardware but can be detected by code generated by the compiler. For example, array subscript range errors are almost never detected by hardware,¹ but they lead to serious errors that often are not noticed until later in the program execution.

Detection of subscript range errors is sometimes required by the language design. For example, the Java language specification requires Java compilers to generate code to check the correctness of every subscript expression (they do not generate such code when it can be determined at compile time that a subscript expression cannot have an out-of-range value, for example, if the subscript is a literal). In C, subscript ranges are not checked because the cost of such checking was (and still is) not believed to be worth the benefit of detecting such errors. In some compilers for some languages, subscript range checking can be selected (if not turned on by default) or turned off (if it is on by default) as desired in the program or in the command that executes the compiler.

The designers of most contemporary languages have included mechanisms that allow programs to react in a standard way to certain run-time errors, as well as other program-detected unusual events. Programs may also be notified when certain events are detected by hardware or system software, so that they also can react to these events.

14.1.1 Basic Concepts

We consider both the errors detected by hardware, such as disk read errors, and unusual conditions, such as end-of-file (which is also detected by hardware), to be exceptions. We further extend the concept of an exception to include errors or unusual conditions that are software-detectable (by either a software interpreter or the user code itself). Accordingly, we define **exception** to be any unusual event, erroneous or not, that is detectable by either hardware or software and that may require special processing.

The special processing that may be required when an exception is detected is called **exception handling**. This processing is done by a code unit or segment called an **exception handler**. An exception is **raised** when its associated event occurs. In some C-based languages, exceptions are said to be *thrown*, rather than *raised*.² Different kinds of exceptions require different exception handlers. Detection of end-of-file nearly always requires some specific program action. But, clearly, that action would not also be appropriate for an array index range error exception. In some cases, the only action is the generation of an error message and an orderly termination of the program.

1. In the 1970s, there were some computers that *did* detect subscript range errors in hardware.
2. C++ was the first C-based language that included exception handling. The word *throw* was used, rather than *raise*, because the standard C library includes a function named `raise`.

The absence of separate or specific exception-handling facilities in a language does not preclude the handling of user-defined, software-detected exceptions. Such an exception detected within a program unit is often handled by the unit's caller. One possible design is to send an auxiliary parameter, which is used as a status variable. The status variable is assigned a value in the called subprogram according to the correctness and/or normalness of the results of its computations. Immediately upon return from the called unit, the caller tests the status variable. If the value indicates that an exception has occurred, the handler, which may reside in the calling unit, can be enacted. Many of the C standard library functions use a variant of this approach: The return values are used as error indicators.

Another possibility is to pass a label parameter to the subprogram. Of course, this approach is possible only in languages that allow labels to be used as parameters. Passing a label allows the called unit to return to a different point in the caller if an exception has occurred. As in the first alternative, the handler is often a segment of the calling unit's code. This is a common use of label parameters in Fortran.

A third possibility is to have the handler defined as a separate subprogram whose name is passed as a parameter to the called unit. In this case, the handler subprogram is provided by the caller, but the called unit calls the handler when an exception is raised. One problem with this approach is that one is required to send a handler subprogram with *every* call to *every* subprogram that takes a handler subprogram as a parameter, whether it is needed or not. Furthermore, to deal with several different kinds of exceptions, several different handler routines would need to be passed, complicating the code.

If it is desirable to handle an exception in the unit in which it is detected, the handler is included as a segment of code in that unit.

There are some definite advantages to having exception handling built into a language. First, without exception handling, the code required to detect error conditions can considerably clutter a program. For example, suppose a subprogram includes expressions that contain 10 references to elements of a matrix named `mat`, and any one of them could have an index out-of-range error. Further suppose that the language does not require index range checking. Without built-in index range checking, every one of these operations may need to be preceded by code to detect a possible index range error. For example, consider the following reference to an element of `mat`, which has 10 rows and 20 columns:

```
if (row >= 0 && row < 10 && col >= 0 && col < 20)
    sum += mat[row][col];
else
    System.out.println("Index range error on mat, row = " +
                       row + " col = " + col);
```

The presence of exception handling in the language would permit the compiler to insert machine code for such checks before every array element access, greatly shortening and simplifying the source program.

Another advantage of language support for exception handling results from exception propagation. Exception propagation allows an exception raised in one program unit to be handled in some other unit in its dynamic or static ancestry. This allows a single exception handler to be used for any number of different program units. This reuse can result in significant savings in development cost, program size, and program complexity.

A language that supports exception handling encourages its users to consider all of the events that could occur during program execution and how they can be handled. This approach is far better than not considering such possibilities and simply hoping nothing will go wrong.

Finally, there are programs in which dealing with nonerroneous but unusual situations can be simplified with exception handling, and in which program structure can become overly convoluted without it.

14.1.2 Design Issues

We now explore some of the design issues for an exception-handling system when it is part of a programming language. Such a system might allow both predefined and user-defined exceptions and exception handlers. Note that predefined exceptions are implicitly raised, whereas user-defined exceptions must be explicitly raised by user code. Consider the following skeletal subprogram that includes an exception-handling mechanism for an implicitly raised exception:

```
void example() {
    . . .
    average = sum / total;
    . . .
    return;
/* Exception handlers */
    when zero_divide {
        average = 0;
        printf("Error-divisor (total) is zero\n");
    }
    . . .
}
```

The exception of division by zero, which is implicitly raised, causes control to transfer to the appropriate handler, which is then executed.

The first design issue for exception handling is how an exception occurrence is bound to an exception handler. This issue occurs on two different levels. On the unit level, there is the question of how the same exception being raised at different points in a unit can be bound to different handlers within the unit. For example, in the example subprogram, there is a handler for a division-by-zero exception that appears to be written to deal with an occurrence of division by zero in a particular statement (the one shown). But suppose the function includes several other expressions with division

operators. For those operators, this handler would probably not be appropriate. So, it should be possible to bind the exceptions that can be raised by particular statements to particular handlers, even though the same exception can be raised by many different statements.

history note

PL/I (ANSI, 1976) pioneered the concept of allowing user programs to be directly involved in exception handling. The language allowed the user to write exception handlers for a long list of language-defined exceptions. Furthermore, PL/I introduced the concept of user-defined exceptions, which allow programs to create software-detected exceptions. These exceptions use the same mechanisms that are used for the built-in exceptions. Since PL/I was designed, a substantial amount of work has been done to design alternative methods of exception handling, and exception-handling mechanisms have been included in a long list of subsequent programming languages.

At a higher level, the binding question arises when there is no exception handler local to the unit in which the exception is raised. In this case, the language designer must decide whether to propagate the exception to some other unit and, if so, where. How this propagation takes place and how far it goes have an important impact on the writability of exception handlers. For example, if handlers must be local, then many handlers must be written, which complicates both the writing and reading of the program. On the other hand, if exceptions are propagated, a single handler might handle the same exception raised in several program units, which may require the handler to be more general than one would prefer.

An issue that is related to the binding of an exception to an exception handler is whether information about the exception is made available to the handler.

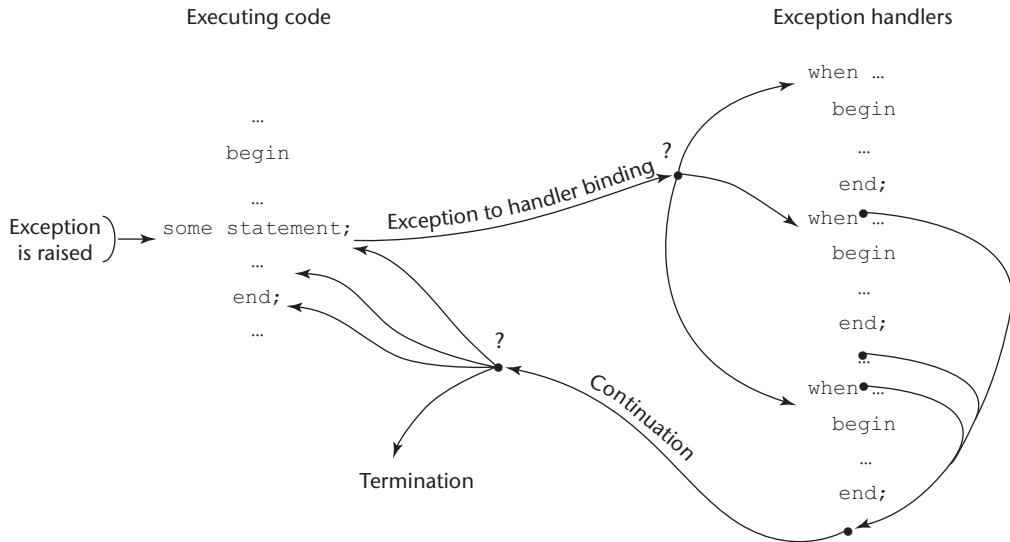
After an exception handler executes, either control can transfer to somewhere in the program outside of the handler code or program execution can simply terminate. We term this the question of control continuation after handler execution, or simply **continuation**. Termination is obviously the simplest choice, and in many error exception conditions, the best. However, in other situations, particularly those associated with unusual but not erroneous events, the choice of continuing execution is best. This design is called **resumption**. In these cases, some conventions must be chosen to determine where execution should continue. It might be the statement that raised the exception, the statement

after the statement that raised the exception, or possibly some other unit. The choice to return to the statement that raised the exception may seem like a good one, but in the case of an error exception, it is useful only if the handler somehow is able to modify the values or operations that caused the exception to be raised. Otherwise, the exception will simply be reraised. The required modification for an error exception is often very difficult to predict. Even when possible, however, it may not be a sound practice. It allows the program to remove the symptom of a problem without removing the cause.

The two issues of binding of exceptions to handlers and continuation are illustrated in Figure 14.1.

When exception handling is included, a subprogram's execution can terminate in two ways: when its execution is complete or when it encounters an exception.³ In some situations, it is necessary to complete some computation

3. Of course, even if the language does not support exception handling, a subprogram could terminate due to a system-detected error.

**Figure 14.1**

Exception-handling control flow

regardless of how subprogram execution terminates. The ability to specify such a computation is called *finalization*. The choice of whether to support finalization is obviously a design issue for exception handling.

Another design issue is the following: If users are allowed to define exceptions, how are these exceptions specified? The usual answer is to require that they be declared in the specification parts of the program units in which they can be raised. The scope of a declared exception is usually the scope of the program unit that contains the declaration.

In the case where a language provides predefined exceptions, several other design issues follow. For example, should the language run-time system provide default handlers for the built-in exceptions, or should the user be required to write handlers for all exceptions? Another question is whether predefined exceptions can be raised explicitly by the user program. This usage can be convenient if there are software-detectable situations in which the user would like to use a predefined handler.

Another issue is whether hardware-detectable errors can be handled by user programs. If not, all exceptions obviously are software detectable. A related question is whether there should be any predefined exceptions. Predefined exceptions are implicitly raised by either hardware or system software.

The exception-handling design issues can be summarized as follows:

- How and where are exception handlers specified, and what is their scope?
- How is an exception occurrence bound to an exception handler?
- Can information about an exception be passed to the handler?

- Where does execution continue, if at all, after an exception handler completes its execution? (This is the question of continuation or resumption.)
- Is some form of finalization provided?
- How are user-defined exceptions specified?
- If there are predefined exceptions, should there be default exception handlers for programs that do not provide their own?
- Can predefined exceptions be explicitly raised?
- Are hardware-detectable errors treated as exceptions that may be handled?
- Are there any predefined exceptions?

We are now in a position to examine the exception-handling facilities of several contemporary programming languages.

14.2 Exception Handling in C++

The exception handling of C++ was accepted by the ANSI C++ standardization committee in 1990 and subsequently found its way into C++ implementations. The design is based in part on the exception handling of CLU, Ada, and ML.

14.2.1 Exception Handlers

C++ uses a special construct that is introduced with the reserved word **try** to specify the scope for exception handlers. A **try** construct includes a compound statement called the **try clause** and a list of exception handlers. The compound statement defines the scope of the following handlers. The general form of this construct is as follows:

```
try {  
    /** Code that might raise an exception  
} catch (formal parameter) {  
    /** A handler body  
}  
  
. . .  
catch(formal parameter) {  
    /** A handler body  
}
```

Each **catch** function is an exception handler. A **catch** function can have only a single formal parameter, which is similar to a formal parameter in a function definition in C++, including the possibility of it being an ellipsis (. . .). A handler with an ellipsis formal parameter is the catch-all handler;

it is enacted for any raised exception if no appropriate handler was found. The formal parameter also can be a naked type specifier, such as **float**, as in a function prototype. In such a case, the only purpose of the formal parameter is to make the handler uniquely identifiable. When information about the exception is to be passed to the handler, the formal parameter includes a variable name that is used for that purpose. Because the class of the parameter can be any user-defined class, the parameter can include as many data members as are necessary. Binding exceptions to handlers are discussed in Section 14.3.2.

In C++, exception handlers can include any C++ code.

14.2.2 Binding Exceptions to Handlers

C++ exceptions are raised only by the explicit statement **throw**, whose general form in EBNF is as follows:

```
throw [expression];
```

The brackets here are metasympols used to specify that the expression is optional. A **throw** without an operand can appear only in a handler. When it appears there, it reraises the exception, which is then handled elsewhere.

The type of the **throw** expression selects the particular handler, which of course must have a “matching” type formal parameter. In this case, *matching* means the following: A handler with a formal parameter of type **T**, **const T**, **T&** (a reference to an object of type **T**), or **const T&** matches a **throw** with an expression of type **T**. In the case where **T** is a class, a handler whose parameter is type **T** or any class that is an ancestor of **T** matches. There are more complicated situations in which a **throw** expression matches a formal parameter, but they are not described here.

An exception raised in a **try** clause causes an immediate end to the execution of the code in that **try** clause. The search for a matching handler begins with the handlers that immediately follow the **try** clause. The matching process is done sequentially on the handlers until a match is found. This means that if any other match precedes an exactly matching handler, the exactly matching handler will not be used. Therefore, handlers for specific exceptions are placed at the top of the list, followed by more generic handlers. The last handler is often one with an ellipsis (...) formal parameter, which matches any exception. This would guarantee that all exceptions are caught.

If an exception is raised in a **try** clause and there is no matching handler associated with that **try** clause, the exception is propagated. If the **try** clause is nested inside another **try** clause, the exception is propagated to the handlers associated with the outer **try** clause. If none of the enclosing **try** clauses yields a matching handler, the exception is propagated to the caller of the function in which it was raised. If the call to the function was not in a **try** clause, the exception is propagated to that function’s caller. If no matching handler is found in the program through this propagation process, the default handler is called. This handler is further discussed in Section 14.2.4.

14.2.3 Continuation

After a handler has completed its execution, control flows to the first statement following the **try** construct (the statement immediately after the last handler in the sequence of handlers of which it is an element). A handler may reraise an exception, using a **throw** without an expression, in which case that exception is propagated.

14.2.4 Other Design Choices

In terms of the design issues summarized in Section 14.1.2, the exception handling of C++ is simple. There are *only* user-defined exceptions, and they are not specified (though they might be declared as new classes). There is a default exception handler, `unexpected`, whose only action is to terminate the program. This handler catches all exceptions not caught by the program. It can be replaced by a user-defined handler. The replacement handler must be a function that returns `void` and takes no parameters. The replacement function is set by assigning its name to `set_terminate`.

A C++ function can list the types of the exceptions (the types of the **throw** expressions) that it could raise. This is done by attaching the reserved word **throw**, followed by a parenthesized list of these types, to the function header. For example,

```
int fun() throw (int, char *) { . . . }
```

specifies that the function `fun` could raise exceptions of type `int` and `char *` but no others. The purpose of the **throw** clause is to notify users of the function what exceptions might be raised by the function. The **throw** clause is in effect a contract between the function and its callers. It guarantees that no other exception will be raised in the function.

If the types in the **throw** clause are classes, then the function can raise any exception that is derived from the listed classes. If a function header has a **throw** clause and raises an exception that is not listed in the **throw** clause and is not derived from a class listed there, the default handler is called. Note that this error cannot be detected at compile time. The list of types in the list may be empty, meaning that the function will not raise any exceptions. If there is no **throw** specification on the header, the function can raise any exception. The list is not part of the function's type.

If a function overrides a function that has a **throw** clause, the overriding function cannot have a **throw** clause with more exceptions than the overridden function.

Although C++ has no predefined exceptions, the standard libraries define and throw exceptions, such as `out_of_range`, which can be thrown by library container classes, and `overflow_error`, which can be thrown by math library functions.

14.2.5 An Example

The following example program illustrates some simple uses of exception handlers in C++. The program computes and prints a distribution of input grades by using an array of counters. The input is a sequence of grades, terminated by a negative number. The negative number raises a `NegativeInputException` exception because the grades must be nonnegative integers. There are 10 categories of grades (0 – 9, 10 – 19, . . . , 90 – 100). The grades themselves are used to compute indexes into an array of counters, one for each grade category. Invalid input grades are detected by trapping indexing errors in the counter array. A grade of 100 is special in the computation of the grade distribution because the categories all have 10 possible grade values, except the highest, which has 11 (90, 91, . . . , 100). (The fact that there are more possible A grades than B's or C's is conclusive evidence of the generosity of teachers.) The grade of 100 is also handled in the same exception handler that is used for invalid input data.

```
// Grade Distribution
// Input: A list of integer values that represent
//        grades, followed by a negative number
// Output: A distribution of grades, as a percentage for
//         each of the categories 0-9, 10-19, . . . ,
//         90-100.
#include <iostream>
int main() { /* Any exception can be raised
    int new_grade,
        index,
        limit_1,
        limit_2,
        freq[10] = {0,0,0,0,0,0,0,0,0,0};
// The exception definition to deal with the end of data
class NegativeInputException {
public:
    NegativeInputException() { /* Constructor
        cout << "End of input data reached" << endl;
    } /*** end of constructor
} /*** end of NegativeInputException class
try {
    while (true) {
        cout << "Please input a grade" << endl;
        if ((cin >> new_grade) < 0) /* End of data
            throw NegativeInputException();
        index = new_grade / 10;
        {try {
            if (index > 9)
                throw new_grade;
```

```

        freq[index]++;
    } /* end of inner try compound
catch(int grade) { /* Handler for index errors
    if (grade == 100)
        freq[9]++;
    else
        cout << "Error -- new grade: " << grade
            << " is out of range" << endl;
    } /* end of catch(int grade)
} /* end of the block for the inner try-catch pair
} /* end of while (1)
} /* end of outer try block
catch(NegativeInputException& e) { /**Handler for
                                /** negative input
    cout << "Limits   Frequency" << endl;
    for (index = 0; index < 10; index++) {
        limit_1 = 10 * index;
        limit_2 = limit_1 + 9;
        if (index == 9)
            limit_2 = 100;
        cout << limit_1 << limit_2 << freq[index] << endl;
    } /* end of for (index = 0)
} /* end of catch (NegativeInputException& e)
} /* end of main

```

This program is meant to illustrate the mechanics of C++ exception handling. Note that the index range exception is often handled in C++ by overloading the indexing operation, which could then raise the exception, rather than the direct detection of the indexing operation with the selection construct used in our example.

14.2.6 Evaluation

One deficiency of exception handling in C++ is that there are no predefined hardware-detectable exceptions that can be handled by the user. Exceptions are connected to handlers through a parameter type in which the formal parameter may be omitted. The type of the formal parameter of a handler determines the condition under which it is called but may have nothing whatsoever to do with the nature of the raised exception. Therefore, the use of predefined types for exceptions certainly does not promote readability. It is much better to define classes for exceptions with meaningful names in a meaningful hierarchy that can be used for defining exceptions. The exception parameter provides a way to pass information about an exception to the exception handler.

14.3 Exception Handling in Java

In Chapter 13, the Java example program includes the use of exception handling with little explanation. This section describes the details of Java's exception-handling capabilities.

Java's exception handling is based on that of C++, but it is designed to be more in line with the object-oriented language paradigm. Furthermore, Java includes a collection of predefined exceptions that are implicitly raised by the Java run-time system.

14.3.1 Classes of Exceptions

All Java exceptions are objects of classes that are descendants of the `Throwable` class. The Java system includes two predefined exception classes that are subclasses of `Throwable`: `Error` and `Exception`. The `Error` class and its descendants are related to errors that are thrown by the Java run-time system, such as running out of heap memory. These exceptions are never thrown by user programs, and they should never be handled there. There are two system-defined direct descendants of `Exception`: `RuntimeException` and `IOException`. As its name indicates, `IOException` is thrown when an error has occurred in an input or output operation, all of which are defined as methods in the various classes defined in the package `java.io`.

There are predefined classes that are descendants of `RuntimeException`. In most cases, `RuntimeException` is thrown when a user program causes an error. For example, `ArrayIndexOutOfBoundsException`, which is defined in `java.util`, is a commonly thrown exception that descends from `RuntimeException`. Another commonly thrown exception that descends from `RuntimeException` is `NullPointerException`.

User programs can define their own exception classes. The convention in Java is that user-defined exceptions are subclasses of `Exception`.

14.3.2 Exception Handlers

The exception handlers of Java have the same form as those of C++, except that every **catch** must have a parameter and the class of the parameter must be a descendant of the predefined class `Throwable`.

The syntax of the **try** construct in Java is exactly as that of C++, except for the **finally** clause described in Section 14.3.6.

14.3.3 Binding Exceptions to Handlers

Throwing an exception is simple. An instance of the exception class is given as the operand of the **throw** statement. For example, suppose we define an exception named `MyException` as

```
class MyException extends Exception {
    public MyException() {}
    public MyException(String message) {
        super (message);
    }
}
```

This exception can be thrown with the following statement:

```
throw new MyException();
```

One of the two constructors we have included in our new class has no parameter and the other has a `String` object parameter that it sends to the superclass (`Exception`), which displays it. Therefore, our new exception could be thrown with

```
throw new MyException  
    ("a message to specify the location of the error");
```

The binding of exceptions to handlers in Java is similar to that of C++. If an exception is thrown in the compound statement of a **try** construct, it is bound to the first handler (**catch** function) immediately following the **try** clause whose parameter is the same class as the thrown object, or an ancestor of it. If a matching handler is found, the **throw** is bound to it and it is executed.

Exceptions can be handled and then rethrown by including a **throw** statement without an operand at the end of the handler. The newly thrown exception will not be handled in the same **try** where it was originally thrown, so looping is not a concern. This rethrowing is usually done when some local action is useful, but further handling by an enclosing **try** clause or a **try** clause in the caller is necessary. A **throw** statement in a handler could also throw some exception other than the one that transferred control to this handler.

To ensure that exceptions that can be thrown in a **try** clause are always handled in a method, a special handler can be written that matches all exceptions that are derived from `Exception` simply by defining the handler with an `Exception` type parameter, as in

```
catch (Exception genericObject) {  
    . . .  
}
```

Because a class name always matches itself or any ancestor class, any class derived from `Exception` matches `Exception`. Of course, such an exception handler should always be placed at the end of the list of handlers, for it will block the use of any handler that follows it in the **try** construct in which it appears. This occurs because the search for a matching handler is sequential, and the search ends when a match is found.

14.3.4 Other Design Choices

As part of its reflection facilities, the Java run-time system stores the class name of every object in the program. The method `getClass` can be used to get an object that stores the class name, which itself can be gotten with the `getName` method. So, we can retrieve the name of the class of the actual parameter

from the **throw** statement that caused the handler's execution. For the handler shown earlier, this is done with

```
genericObject.getClass().getName()
```

In addition, the message associated with the parameter object, which is created by the constructor, can be gotten with

```
genericObject.getMessage()
```

Furthermore, in the case of user-defined exceptions, the thrown object could include any number of data fields that might be useful in the handler.

The **throws** clause of Java has the appearance and placement (in a program) that is similar to that of the **throw** specification of C++. However, the semantics of **throws** is somewhat different from that of the C++ **throw** clause.

The appearance of an exception class name in the **throws** clause of a Java method specifies that that exception class or any of its descendant exception classes can be thrown but not handled by the method. For example, when a method specifies that it can throw `IOException`, it means it can throw an `IOException` object or an object of any of its descendant classes, such as `EOFException`, and it does not handle the exception it throws.

Exceptions of class `Error` and `RuntimeException` and their descendants are called **unchecked exceptions**. All other exceptions are called **checked exceptions**. Unchecked exceptions are never a concern of the compiler. However, the compiler ensures that all checked exceptions a method can throw are either listed in its **throws** clause or handled in the method. Note that checking this at compile time differs from C++, in which it is done at run time. The reason why exceptions of the classes `Error` and `RuntimeException` and their descendants are unchecked is that any method could throw them. A program can catch unchecked exceptions, but it is not required.

As is the case with C++, a method cannot declare more exceptions in its **throws** clause than the method it overrides, though it may declare fewer. So if a method has no **throws** clause, neither can any method that overrides it. A method can throw any exception listed in its **throws** clause, along with any of the descendant classes of those exceptions.

A method that does not directly throw a particular exception, but calls another method that could throw that exception, must list the exception in its **throws** clause. This is the reason the `buildDist` method (in the example in the next subsection), which uses the `readLine` method, must specify `IOException` in the **throws** clause of its header.

A method that does not include a **throws** clause cannot propagate any checked exception. Recall that in C++, a function without a **throw** clause can throw *any* exception.

A method that calls a method that lists a particular checked exception in its **throws** clause has three alternatives for dealing with that exception: First, it can catch the exception and handle it. Second, it can catch the exception and throw an exception that is listed in its own **throws** clause. Third, it could

declare the exception in its own **throws** clause and not handle it, which effectively propagates the exception to an enclosing **try** clause, if there is one, or to the method's caller, if there is no enclosing **try** clause.

There are no default exception handlers, and it is not possible to disable exceptions. Continuation in Java is exactly as in C++.

14.3.5 An Example

Following is the Java program with the capabilities of the C++ program in Section 14.2.5:

```
// Grade Distribution
// Input: A list of integer values that represent
//        grades, followed by a negative number
// Output: A distribution of grades, as a percentage for
//        each of the categories 0-9, 10-19, . . . ,
//        90-100.
import java.io.*;
// The exception definition to deal with the end of data
class NegativeInputException extends Exception {
    public NegativeInputException() {
        System.out.println("End of input data reached");
    } /** end of constructor
} /** end of NegativeInputException class

class GradeDist {
    int newGrade,
        index,
        limit_1,
        limit_2;
    int [] freq = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

void buildDist() throws IOException {
    DataInputStream in = new DataInputStream(System.in);
    try {
        while (true) {
            System.out.println("Please input a grade");
            newGrade = Integer.parseInt(in.readLine());
            if (newGrade < 0)
                throw new NegativeInputException();
            index = newGrade / 10;
            try {
                freq[index]++;
            } /** end of inner try clause
        catch (ArrayIndexOutOfBoundsException e) {
            if (newGrade == 100)
                freq [9]++;
            else
                System.out.println("Error - new grade: " +
                                   newGrade + " is out of range");
        }
    }
}
```

```

    } /** end of catch (ArrayIndex. . .
  } /** end of while (true) . . .
} /** end of outer try clause
catch(NegativeInputException e) {
    System.out.println ("\nLimits      Frequency\n");
    for (index = 0; index < 10; index++) {
        limit_1 = 10 * index;
        limit_2 = limit_1 + 9;
        if (index == 9)
            limit_2 = 100;
        System.out.println(" " + limit_1 + " - " +
                           limit_2 + "      " + freq [index]);
    } /** end of for (index = 0; ...
  } /** end of catch (NegativeInputException ...
} /** end of method buildDist

```

The exception for a negative input, `NegativeInputException`, is defined in the program. Its constructor displays a message when an object of the class is created. Its handler produces the output of the method. `ArrayIndexOutOfBoundsException` is a predefined unchecked exception that is thrown by the Java run-time system. In both of these cases, the handler does not include an object name in its parameter. In neither case would a name serve any purpose. Although all handlers get objects as parameters, they often are not useful.

14.3.6 The **finally** Clause

There are some situations in which a process must be executed regardless of whether a **try** clause throws an exception or the exception is handled in the method. One example of such a situation is a file that must be closed. Another is if the method has some external resource that must be freed in the method regardless of how the execution of the method terminates. The **finally** clause was designed for these kinds of needs. A **finally** clause is placed at the end of the list of handlers just after a complete **try** construct. In general, the **try** construct and its **finally** clause appear as

```

try {
    . . .
}
catch (. . .) {
    . . .
}
. . . /** More handlers
finally {
    . . .
}

```

The semantics of this construct is as follows: If the **try** clause throws no exceptions, the **finally** clause is executed before execution continues after the **try** construct. If the **try** clause throws an exception and it is caught by a following

handler, the **finally** clause is executed after the handler completes its execution. If the **try** clause throws an exception but it is not caught by a handler following the **try** construct, the **finally** clause is executed before the exception is propagated.

A **try** construct with no exception handlers can be followed by a **finally** clause. This makes sense, of course, only if the compound statement has a **throw**, **break**, **continue**, or **return** statement. Its purpose in these cases is the same as when it is used with exception handling. For example, consider the following:

```
try {
    for (index = 0; index < 100; index++) {
        . . .
        if ( . . . ) {
            return;
        } /** end of if
    } /** end of for
} /** end of try clause
finally {
    . . .
} /** end of try construct
```

The **finally** clause here will be executed, regardless of whether the **return** terminates the loop or it ends normally.

14.3.7 Assertions

In the discussion of Plankalkül in Chapter 2, we mentioned that it included assertions. Assertions were added to Java in version 1.4. To use them, it is necessary to enable them by running the program with the `enableassertions` (or `ea`) flag, as in

```
java -enableassertions MyProgram
```

There are two possible forms of the **assert** statement:

```
assert condition;
assert condition : expression;
```

In the first case, the condition is tested when execution reaches the **assert**. If the condition evaluates to true, nothing happens. If it evaluates to false, the `AssertionError` exception is thrown. In the second case, the action is the same, except that the value of the expression is passed to the `AssertionError` constructor as a string and becomes debugging output.

The **assert** statement is used for defensive programming. A program may be written with many **assert** statements, which ensure that the program's computation is on track to produce correct results. Many programmers put in such checks when they write a program, as an aid to debugging, even though the language they are using does not support assertions. When the program

is sufficiently tested, these checks are removed. The advantage of **assert** statements, which have the same purpose, is that they can be disabled without removing them from the program. This saves the effort of removing them and also allows their use during subsequent program maintenance.

14.3.8 Evaluation

The Java mechanisms for exception handling are an improvement over the C++ version on which they are based.

First, a C++ program can throw any type defined in the program or by the system. In Java, only objects that are instances of `Throwable` or some class that descends from it can be thrown. This separates the objects that can be thrown from all of the other objects (and nonobjects) that inhabit a program. What significance can be attached to an exception that causes an **int** value to be thrown?

Second, a C++ program unit that does not include a **throw** clause can throw any exception, which tells the reader nothing. A Java method that does not include a **throws** clause cannot throw any checked exception that it does not handle. Therefore, the reader of a Java method knows from its header what exceptions it could throw but does not handle. A C++ compiler ignores **throw** clauses, but a Java compiler ensures that all exceptions that a method can throw are listed in its **throws** clause.

Third, the **finally** clause is a useful addition. It allows cleanup kinds of actions to take place regardless of how a compound statement terminated.

Finally, the Java run-time system implicitly throws a variety of predefined exceptions, such as for array indices out of range and dereferencing null references, which can be handled by any user program. A C++ program can handle only those exceptions that it explicitly throws (or that are thrown by library classes it uses).

C# includes exception-handling constructs that are very much like those of Java, except that C# does not have a **throws** clause.

14.4 Exception Handling in Python and Ruby

This section provides brief overviews of the exception-handling mechanisms of Python and Ruby.

14.4.1 Python

In Python, exceptions are objects. The base class of all exception classes is `BaseException`, from which the `Exception` class is derived. `BaseException` provides some services that are useful for all exception classes, but it is not usually directly subclassed. All predefined exception classes are derived from `Exception` and user-defined exception classes also are derived from it. The most commonly used predefined subclasses of `Exception` are `ArithmeticError`,

whose primary subclasses are `OverflowError`, `ZeroDivisionError`, and `FloatingPointError`, and `LookupError`, whose main subclasses are `IndexError` and `KeyError`.

The statements for dealing with exceptions are similar to those of Java. The general form of a **try** construct is as follows:

```
try:
    The try block (the range of statements to be watched for exceptions)
except Exception1:
    Handler for Exception1
except Exception2:
    Handler for Exception2
...
else:
    The else block (what to do when no exception is raised)
finally:
    The finally block (what must be done regardless of what happened)
```

Both the **else** and the **finally** clauses are optional.

One difference between handlers in Java and Python is that Python uses **except** to introduce them, rather than **catch**. The **else** clause is executed if no exception is raised in the **try** block. The **finally** clause has the same semantics as its counterpart in Java: If an exception is raised in the **try** block but is not handled by an immediately following handler, the exception is propagated after the **finally** block is executed. Because a handler handles its named exception, as well as all subclasses of that exception, a handler that names `Exception` handles all predefined and user-defined exceptions.

An unhandled exception is propagated to progressively larger enclosing **try** constructs, searching for an appropriate handler. If none is found, the exception is propagated to the function's caller, again searching for a handler in a nesting **try** construct. If no handler is found at any level, the default handler is called, which produces an error message and a stack trace and terminates the program.

The **raise** statement of Python is similar to the **throw** statement of Java and C++. The parameter for **raise** is the class name of the exception to be raised. For example, we could have the following:

```
raise IndexError
```

This statement implicitly creates an instance of the named class, `IndexError`.

An exception handler can gain access to the object of the raised exception by providing an **as** clause and a variable name, as in the following:

```
except Exception as ex_obj:
```

This is a universal handler, as it handles all exceptions. The exception object can be printed with a **print** statement in the handler, which produces the

message of the object. For example, if the exception was `ZeroDivisionError`, the message would be `division by zero`.

Python's **assert** statement provides a mechanism for making some exception handling optional. The general form of **assert** is as follows:

```
assert test, data
```

In this statement, `test` is a Boolean flag or expression and `data` is the value that is sent to the constructor for the exception object to be raised. The meaning of this statement, which optionally raises the `AssertionError` exception, can be described with the following code:

```
if __debug__:
    if not test:
        raise AssertionError(data)
```

`__debug__` is a predefined flag that is set to `True` unless the `-0` flag is used on the command that runs the program. This allows one to disable all **assert** statements for a particular run of the program. If an `AssertionError` exception is not handled by the program, like other unhandled exceptions, it terminates the program after using the default handler.

Python does not have an equivalent to the **throws** clause of Java.

14.4.2 Ruby

Like Python, Ruby exceptions are objects and it has a large collection of predefined exception classes. All of the exceptions that are handled by application programs are either objects of the `StandardError` class or a class that descends from it. `StandardError` is derived from `Exception`, which provides two useful methods to all its descendants. These are `message`, which returns the human-readable error message, and `backtrace`, which returns a stack trace starting from the method where the exception was raised. Some of the predefined subclasses of `StandardError` are `ArgumentError`, `IndexError`, `IOError`, and `ZeroDivisionError`.

Exceptions are explicitly raised with the `raise` method. `raise` is often called with a string parameter. In this case, it raises a new `RuntimeError` object with the string as its message. For example, we could have the following:

```
raise "bad parameter" if count == 0
```

`raise` could also have two parameters, the first of which would be an object of an exception class. The `exception` method of this object is called and the returned `Exception` object is raised. In this case, the second parameter would be the string message to be displayed. For example, we could have the following:

```
raise TypeError, "Float parameter expected"
if not param.is_a? Float
```

An exception handler is specified with a **rescue** clause, which is attached to a statement. To attach an exception handler to a segment of code, the code is placed in a **begin-end** block. The **rescue** clause is placed in the block after the code of the block. In general, this appears as in the following:

```
begin
  The sequence of statements in the block
rescue
  The handler
end
```

A **begin-end** block can include an **else** clause and/or an **ensure** clause. The **else** clause is exactly like that of Python. The **ensure** clause is exactly like a **finally** clause. A method can act as a container for exception handling in place of a **begin-end** block.

In a clear departure from most other languages, Ruby allows a segment of code that raised an exception to be rerun after the exception is handled. This is specified with a **retry** statement at the end of the handler.

14.5 Introduction to Event Handling

Event handling is similar to exception handling. In both cases, the handlers are implicitly called by the occurrence of something, either an exception or an event. While exceptions can be raised either explicitly by user code or implicitly by hardware or a software interpreter, events are created by external actions, such as user interactions through a graphical user interface (GUI). In this section, the fundamentals of event handling, which are less complex than those of exception handling, are introduced.

In conventional (non-event-driven) programming, the program code itself specifies the order in which that code is executed, although the order is usually affected by the program's input data. In event-driven programming, parts of the program are executed at completely unpredictable times, often triggered by user interactions with the executing program.

The particular kind of event handling discussed in this chapter is related to GUIs. Therefore, most of the events are caused by user interactions through graphical objects or components, often called *widgets*. The most common widgets are buttons. Implementing reactions to user interactions with GUI components is the most common form of event handling.

An **event** is a notification that something specific has occurred, such as a mouse click on a graphical button. Strictly speaking, an event is an object that is implicitly created by the run-time system in response to a user action, at least in the context in which event handling is being discussed here.

An **event handler** is a segment of code that is executed in response to the appearance of an event. Event handlers enable a program to be responsive to user actions.

Although event-driven programming was being used long before GUIs appeared, it has become a widely used programming methodology only in response to the popularity of these interfaces. As an example, consider the GUIs presented to users of Web browsers. Many Web documents presented to browser users are now dynamic. Such a document may present an order form to the user, who chooses the merchandise by clicking buttons. The required internal computations associated with these button clicks are performed by event handlers that react to the click events.

Another common use of event handlers is to check for simple errors and omissions in the elements of a form, either when they are changed or when the form is submitted to the Web server for processing. Using event handling on the browser to check the validity of form data saves the time of sending that data to the server, where their correctness then must be checked by a server-resident program or script before they can be processed. This kind of event-driven programming is often done using a client-side scripting language, such as JavaScript.

14.6 Event Handling with Java

In addition to Web applications, non-Web Java applications can present GUIs to users. GUIs in Java applications are discussed in this section.

The initial version of Java provided a somewhat primitive form of support for GUI components. In version 1.2 of the language, released in late 1998, a new collection of components was added. These were collectively called Swing.

14.6.1 Java Swing GUI Components⁴

The Swing collection of classes and interfaces, defined in `javax.swing`, includes GUI components, or widgets. Because our interest here is event handling, not GUI components, we discuss only two kinds of widgets: text boxes and radio buttons.

A text box is an object of class `JTextField`. The simplest `JTextField` constructor takes a single parameter, the length of the box in characters. For example,

```
JTextField name = new JTextField(32);
```

The `JTextField` constructor can also take a literal string as an optional first parameter. This string parameter, when present, is displayed as the initial contents of the text box.

Radio buttons are special buttons that are placed in a button group container. A button group is an object of class `ButtonGroup`, whose constructor takes no parameters. In a radio button group, only one button can be pressed at a time. If any button in the group becomes pressed, the previously pressed button is implicitly unpressed. The `JRadioButton` constructor, used for creating radio

4. Over the last few years, Swing is slowly being replaced by a new GUI toolset named JAWAFX.

buttons, takes two parameters: a label and the initial state of the radio button (`true` or `false`, for pressed and not pressed, respectively). If one radio button in a group is initially set to pressed, the others in the group default to unpressed. After the radio buttons are created, they are placed in their button group with the `add` method of the group object. Consider the following example:

```
ButtonGroup payment = new ButtonGroup();
JRadioButton box1 = new JRadioButton("Visa", true);
JRadioButton box2 = new JRadioButton("Master Charge");
JRadioButton box3 = new JRadioButton("Discover");
payment.add(box1);
payment.add(box2);
payment.add(box3);
```

A `JFrame` object is a frame, which is displayed as a separate window. The `JFrame` class defines the data and methods that are needed for frames. So, a class that uses a frame can be a subclass of `JFrame`. A `JFrame` has several layers called **panes**. We are interested in just one of those layers, the content pane. Components of a GUI are placed in a `JPanel` object (a panel), which is used to organize and define the layout of the components. A frame is created and the panel containing the components is added to that frame's content pane.

Predefined graphic objects, such as GUI components, are placed directly in a panel. The following creates the panel object used in the following discussion of components:

```
JPanel myPanel = new JPanel();
```

After the components have been created with constructors, they are placed in the panel with the `add` method, as in

```
myPanel.add(button1);
```

14.6.2 The Java Event Model

When a user interacts with a GUI component, for example by clicking a button, the component creates an event object and calls an event handler through an object called an event listener, passing the event object. The event handler provides the associated actions. GUI components are event generators. In Java, events are connected to event handlers through **event listeners**. Event listeners are connected to event generators through event listener registration. Listener registration is done with a method of the class that implements the listener interface, as described later in this section. Only event listeners that are registered for a specific event are notified when that event occurs.

The listener method that receives the message implements an event handler. To make the event-handling methods conform to a standard protocol, an interface is used. An interface prescribes standard method protocols but does not provide implementations of those methods.

A class that needs to implement an event handler must implement an interface for the listener for that handler. There are several classes of events and listener interfaces. One class of events is `ItemEvent`, which is associated with the event of clicking a checkbox or a radio button, or selecting a list item. The `ItemListener` interface includes the protocol of a method, `itemStateChanged`, which is the handler for `ItemEvent` events. So, to provide an action that is triggered by a radio button click, the interface `ItemListener` must be implemented, which requires a definition of the method, `itemStateChanged`.

As stated previously, the connection of a component to an event listener is made with a method of the class that implements the listener interface. For example, because `ItemEvent` is the class name of event objects created by user actions on radio buttons, the `addItemListener` method is used to register a listener for radio buttons. The listener for button events created in a panel could be implemented in the panel or a subclass of `JPanel`. So, for a radio button named `button1` in a panel named `myPanel` that implements the `ItemEvent` event handler for buttons, we would register the listener with the following statement:

```
button1.addItemListener(this);
```

Each event handler method receives an event parameter, which provides information about the event. Event classes have methods to access that information. For example, when called through a radio button, the `isSelected` method returns `true` or `false`, depending on whether the button was on or off (pressed or not pressed), respectively.

All the event-related classes are in the `java.awt.event` package, so it is imported to any class that uses events.

The following is an example application, `RadioB`, that illustrates the use of events and event handling. This application constructs radio buttons that control the font style of the contents of a text field. It creates a `Font` object for each of four font styles. Each of these has a radio button to enable the user to select the font style.

The purpose of this example is to show how events raised by GUI components can be handled to change the output display of the program dynamically. Because of our narrow focus on event handling, some parts of this program are not explained here.

```
/* RadioB.java
   An example to illustrate event handling with interactive
   radio buttons that control the font style of a textfield
*/
package radiob;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class RadioB extends JPanel implements
    ItemListener {
    private JTextField text;
```

```

    private Font plainFont, boldFont, italicFont,
        boldItalicFont;
    private JRadioButton plain, bold, italic, boldItalic;
    private ButtonGroup radioButtons;

    // The constructor method is where the display is initially
    // built
    public RadioB() {

        // Create the test text string and set its font
        text = new JTextField(
            "In what font style should I appear?", 25);
        text.setFont(plainFont);

        // Create radio buttons for the fonts and add them to
        // a new button group
        plain = new JRadioButton("Plain", true);
        bold = new JRadioButton("Bold");
        italic = new JRadioButton("Italic");
        boldItalic = new JRadioButton("Bold Italic");
        radioButtons = new ButtonGroup();
        radioButtons.add(plain);
        radioButtons.add(bold);
        radioButtons.add(italic);
        radioButtons.add(boldItalic);

        // Create a panel and put the text and the radio
        // buttons in it; then add the panel to the frame
        JPanel radioPanel = new JPanel();
        radioPanel.add(text);
        radioPanel.add(plain);
        radioPanel.add(bold);
        radioPanel.add(italic);
        radioPanel.add(boldItalic);
        add(radioPanel, BorderLayout.LINE_START);

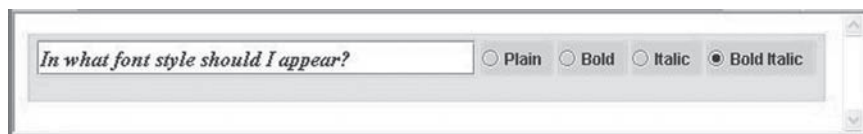
        // Register the event handlers
        plain.addItemListener(this);
        bold.addItemListener(this);
        italic.addItemListener(this);
        boldItalic.addItemListener(this);

        // Create the fonts
        plainFont = new Font("Serif", Font.PLAIN, 16);
        boldFont = new Font("Serif", Font.BOLD, 16);
        italicFont = new Font("Serif", Font.ITALIC, 16);
        boldItalicFont = new Font("Serif", Font.BOLD +
            Font.ITALIC, 16);
    } // End of the constructor for RadioB

```

Figure 14.2

Output of
RadioB.java
Source: Java radio applet
screenshot.



```
// The event handler
public void itemStateChanged (ItemEvent e) {

// Determine which button is on and set the font
// accordingly
    if (plain.isSelected())
        text.setFont(plainFont);
    else if (bold.isSelected())
        text.setFont(boldFont);
    else if (italic.isSelected())
        text.setFont(italicFont);
    else if (boldItalic.isSelected())
        text.setFont(boldItalicFont);
} // End of itemStateChanged

// The main method
public static void main(String[] args) {
// Create the window frame
    JFrame myFrame = new JFrame(" Radio button
                                example");

// Create the content pane and set it to the frame
    JComponent myContentPane = new RadioB();
    myContentPane.setOpaque(true);
    myFrame.setContentPane(myContentPane);

// Display the window.
    myFrame.pack();
    myFrame.setVisible(true);
}
} // End of RadioB
```

The RadioB.java application produces the screen shown in Figure 14.2.

14.7 Event Handling in C#

Event handling in C# (and in the other .NET languages) is similar to that of Java. .NET provides two approaches to creating GUIs in applications, the original Windows Forms and the more recent Windows Presentation Foundation. The latter is the more sophisticated and complex of the two. Because

our interest is just in event handling, we will use the simpler Windows Forms to discuss our subject.

Using Windows Forms, a C# application that constructs a GUI is created by subclassing the `Form` predefined class, which is defined in the `System.Windows.Forms` namespace. This class implicitly provides a window to contain our components. There is no need to build frames or panels explicitly.

Text can be placed in a `Label` object and radio buttons are objects of the `RadioButton` class. The size of a `Label` object is not explicitly specified in the constructor; rather it can be specified by setting the `AutoSize` data member of the `Label` object to `true`, which sets the size according to what is placed in it.

Components can be placed at a particular location in the window by assigning a new `Point` object to the `Location` property of the component. The `Point` class is defined in the `System.Drawing` namespace. The `Point` constructor takes two parameters, which are the coordinates of the object in pixels. For example, `Point(100, 200)` is a position that is 100 pixels from the left edge of the window and 200 pixels from the top. The label of a component is set by assigning a string literal to the `Text` property of the component. After creating a component, it is added to the form window by sending it to the `Add` method of the `Controls` subclass of the form. Therefore, the following code creates a radio button with the label `Plain` at the (100, 300) position in the output window:

```
private RadioButton plain = new RadioButton();
plain.Location = new Point(100, 300);
plain.Text = "Plain";
Controls.Add(plain);
```

All C# event handlers have the same protocol: the return type is `void` and the two parameters are of types `object` and `EventArgs`. Neither of the parameters needs to be used for a simple situation. An event handler method can have any name. A radio button is tested to determine whether it is clicked with the Boolean `Checked` property of the button. Consider the following skeletal example of an event handler:

```
private void rb_CheckedChanged (object o, EventArgs e){
    if (plain.Checked) . . .
    . . .
}
```

To register an event, a new `EventHandler` object must be created. The constructor for this class is sent the name of the handler method. The new object is added to the predefined delegate for the event on the component object (using the `+=` assignment operator). For example, when a radio button changes from unchecked to checked, the `CheckedChanged` event is raised and the handlers registered on the associated delegate, which is referenced by the name of the event, are called. If the event handler is named `rb_CheckedChanged`, the following statement would register the handler for the `CheckedChanged` event on the radio button `plain`:

```
plain.CheckedChanged +=
    new EventHandler(rb_CheckedChanged);
```

Following is the `RadioB` example from Section 14.6 rewritten in C#. Once again, because our focus is on event handling, we do not explain all of the details of the program.

```
// RadioB.cs
// An example to illustrate event handling with
//   interactive radio buttons that control the font
//   style of a string of text

namespace RadioB {

    using System;
    using System.Drawing;
    using System.Windows.Forms;
    public class RadioB : Form {
        private Label text = new Label();
        private RadioButton plain = new RadioButton();
        private RadioButton bold = new RadioButton();
        private RadioButton italic = new RadioButton();
        private RadioButton boldItalic = new RadioButton();

        // Constructor for RadioB
        public RadioB() {

            // Initialize the attributes of the text and radio
            // buttons
            text.AutoSize = true;
            text.Text = "In what font style should I appear?";
            plain.Location = new Point(220,0);
            plain.Text = "Plain";
            plain.Checked = true;
            bold.Location = new Point(350, 0);
            bold.Text = "Bold";
            italic.Location = new Point(480, 0);
            italic.Text = "Italics";
            boldItalic.Location = new Point(610, 0);
            boldItalic.Text = "Bold/Italics";

            // Add the text and the radio buttons to the form
            Controls.Add(text);
            Controls.Add(plain);
            Controls.Add(bold);
            Controls.Add(italic);
            Controls.Add(boldItalic);

            // Register the event handler for the radio buttons
            plain.CheckedChanged +=
                new EventHandler(rb_CheckedChanged);
```

```

        bold.CheckedChanged +=
            new EventHandler(rb_CheckedChanged);
        italic.CheckedChanged +=
            new EventHandler(rb_CheckedChanged);
        boldItalic.CheckedChanged +=
            new EventHandler(rb_CheckedChanged);
    }

    // The main method is where execution begins
    static void Main() {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault (false);
        Application.Run(new RadioB());
    }
    // The event handler

    private void rb_CheckedChanged (object o,
                                    EventArgs e) {

        // Determine which button is on and set the font
        // accordingly
        if (plain.Checked)
            text.Font =
                new Font( text.Font.Name, text.Font.Size,
                           FontStyle.Regular);
        if (bold.Checked)
            text.Font =
                new Font(text.Font.Name, text.Font.Size,
                           FontStyle.Bold);
        if (italic.Checked)
            text.Font =
                new Font(text.Font.Name, text.Font.Size,
                           FontStyle.Italic);
        if (boldItalic.Checked)
            text.Font =
                new Font(text.Font.Name, text.Font.Size,
                           FontStyle.Italic ^ FontStyle.Bold);
    } // End of radioButton_CheckedChanged

} // End of RadioB
}

```

The output from this program is exactly like that shown in Figure 14.2.

S U M M A R Y

C++ includes no predefined exceptions (except those defined in the standard library). C++ exceptions are objects of a primitive type, a predefined class, or a user-defined class. Exceptions are bound to handlers by connecting the type of

the expression in the **throw** statement to that of the formal parameter of the handler. Handlers all have the same name—**catch**. The C++ **throw** clause of a method lists the types of exceptions that the method could throw.

Java exceptions are objects whose ancestry must trace back to a class that descends from the `Throwable` class. There are two categories of exceptions—checked and unchecked. Checked exceptions are a concern for the user program and the compiler. Unchecked exceptions can occur anywhere and are often ignored by user programs.

The Java **throws** clause of a method lists the checked exceptions that it could throw and does not handle. It must include exceptions that methods it calls could raise and propagate back to its caller.

The Java **finally** clause provides a mechanism for guaranteeing that some code will be executed regardless of how the execution of a **try** compound terminates.

Java now includes an **assert** statement, which facilitates defensive programming.

Python's exception handling is similar to that of Java, although it adds the **else** clause to the **try** construct. Also, it uses **except** clauses rather than **catch** clauses to define handlers and **raise** instead of **throw**. Access to the data of an exception object is gained by assigning the object to a variable with an **as** clause. Python's **assert** statement is a conditional **raise**.

Exception handling in Ruby is similar to that of Python. Every exception class has two methods, `message` and `backtrace`. Exceptions are often raised with a **raise** statement with a single string parameter. This creates a new `RuntimeError` object with the string as its message. The **raise** statement can be made conditional by adding a conditional expression to it. The scope of exception handlers usually is specified with a **begin-end** block. Handlers are defined in **rescue** clauses. A **begin-end** block can include an **else** clause and an **ensure** clause, which is like the **finally** clause of Python and Java.

An event is a notification that something has occurred that requires special processing. Events are often created by user interactions with a program through a graphical user interface. Java event handlers are called through event listeners. An event listener must be registered for an event if it is to be notified when the event occurs. Two of the most commonly used event listeners interfaces are `actionPerformed` and `itemStateChanged`.

Windows Forms is the original approach to building GUI components and handling events in .NET languages. A C# application builds a GUI in this approach by subclassing the `Form` class. All .NET event handlers use the same protocol. Event handlers are registered by creating an `EventHandler` object and assigning it to the predefined delegate associated with the GUI object that can raise the event.

BIBLIOGRAPHIC NOTES

One of the most important papers on exception handling that is not connected with a particular programming language is the work by Goodenough (1975). The problems with the PL/I design for exception handling are covered in

MacLaren (1977). Exception handling in C++ is described by Stroustrup (1997). Exception handling in Java is described by Campione et al. (2001).

REVIEW QUESTIONS

1. Define *exception*, *exception handler*, *raising an exception*, *continuation*, *finalization*, and *built-in exception*.
2. What are the two alternatives for designing continuation?
3. What are the advantages of having support for exception handling built in to a language?
4. What are the design issues for exception handling?
5. What does it mean for an exception to be bound to an exception handler?
6. What is the name of all C++ exception handlers?
7. How can exceptions be explicitly raised in C++?
8. How are exceptions bound to handlers in C++?
9. How can an exception handler be written in C++ so that it handles any exception?
10. Where does execution control go when a C++ exception handler has completed its execution?
11. Does C++ include built-in exceptions?
12. Why is the raising of an exception in C++ not called *raise*?
13. What is the root class of all Java exception classes?
14. What is the parent class of most Java user-defined exception classes?
15. How can an exception handler be written in Java so that it handles any exception?
16. What are the differences between a C++ **throw** specification and a Java **throws** clause?
17. What is the difference between checked and unchecked exceptions in Java?
18. How can an exception handler be written in Java so that it handles any exception?
19. What is the purpose of the Java **finally** clause?
20. What advantage do language-defined assertions have over simple **if-then** constructs?
21. Explain what an **else** block in Python does.
22. What is the purpose of an **as** clause in Python?
23. Explain what an **assert** statement in Python does.
24. What does the `message` method of Ruby's `StandardError` class do?

25. What exactly happens when a **raise** statement with a string parameter is executed?
26. What exactly does a Ruby **ensure** clause do?
27. In what ways are exception handling and event handling related?
28. Define event and event handler.
29. What is event-driven programming?
30. What is the purpose of a Java `JFrame`?
31. What is the purpose of a Java `JPanel`?
32. What object is often used as the event listener in Java GUI applications?
33. What is the origin of the protocol for an event handler in Java?
34. What method is used to register an event handler in Java?
35. Using .NET's Windows Forms, what namespace is required to build a GUI for a C# application?
36. How is a component positioned in a form using Windows Forms?
37. What is the protocol of a .NET event handler?
38. What class of object must be created to register a .NET event handler?
39. What role do delegates play in the process of registering event handlers?

PROBLEM SET

1. What did the designers of C get in return for not requiring subscript range checking?
2. Describe three approaches to exception handling in languages that do not provide direct support for it.
3. From textbooks on the PL/I and Ada programming languages, look up the respective sets of built-in exceptions. Do a comparative evaluation of the two, considering both completeness and flexibility.
4. From a textbook on COBOL, determine how exception handling is done in COBOL programs.
5. In languages without exception-handling facilities, it is common to have most subprograms include an "error" parameter, which can be set to some value representing "OK" or some other value representing "error in procedure." What advantage does a linguistic exception-handling facility like that of Java have over this method?
6. In a language without exception-handling facilities, we could send an error-handling procedure as a parameter to each procedure that can detect errors that must be handled. What disadvantages are there to this method?
7. Compare the methods suggested in Problems 5 and 6. Which do you think is better and why?

8. Write a comparative analysis of the **throw** clause of C++ and the **throws** clause of Java.
9. Consider the following C++ skeletal program:

```

class Big {
    int i;
    float f;
    void fun1() throw int {
        . . .
        try {
            . . .
            throw i;
            . . .
            throw f;
            . . .
        }
        catch(float) { . . . }
        . . .
    }
}

class Small {
    int j;
    float g;
    void fun2() throw float {
        . . .
        try {
            . . .
            try {
                Big.fun1();
                . . .
                throw j;
                . . .
                throw g;
                . . .
            }
            catch(int) { . . . }
            . . .
        }
        catch(float) { . . . }
    }
}

```

In each of the four **throw** statements, where is the exception handled? Note that `fun1` is called from `fun2` in class `Small`.

10. Write a detailed comparison of the exception-handling capabilities of C++ and those of Java.
11. With the help of a book on ML, write a detailed comparison of the exception-handling capabilities of ML and those of Java.
12. Summarize the arguments in favor of the termination and resumption models of continuation.

PROGRAMMING EXERCISES

1. Suppose you are writing a C++ function that has three alternative approaches for accomplishing its requirements. Write a skeletal version of this function so that if the first alternative raises any exception, the second is tried, and if the second alternative raises any exception, the third is executed. Write the code as if the three methods were procedures named `alt1`, `alt2`, and `alt3`.
2. Write a Java program that inputs a list of integer values in the range of -100 to 100 from the keyboard and computes the sum of the squares of the input values. This program must use exception handling to ensure that the input values are in range and are legal integers, to handle the error of the sum of the squares becoming larger than a standard `Integer` variable can store, and to detect end-of-file and use it to cause the output of the result. In the case of overflow of the sum, an error message must be printed and the program terminated.
3. Write a C++ program for the specification of Programming Exercise 2.
4. Revise the Java program of Section 14.3.5 to use `EOFException` to detect the end of the input.
5. Rewrite the Java code of Section 14.3.6 that uses a **finally** clause in C++.

